

AI DEBUGGER PRO

AI Debugger Pro: Security Threat Model for Human-Governed AI-Assisted Repair



ABSTRACT

AI-assisted debuggers join three unusually sensitive activities: executing untrusted code, collecting diagnostic context, and asking a probabilistic model to propose source changes. This paper presents a repository-grounded security threat model for AI Debugger Pro, a local desktop prototype that executes Python, C, C++, and Java; captures failures; requests structured diagnoses; previews repairs; requires explicit approval; and reruns approved changes. The analysis separates four trust domains: the desktop host, ephemeral execution environment, model-service boundary, and durable local records. It identifies assets, adversaries, entry points, abuse paths, implemented controls, residual risks, and prioritized mitigations. The present Docker backend provides meaningful defense in depth through network denial, resource limits, a read-only container root, capability dropping, no-new-privileges, an ephemeral workspace, and a deliberately minimal inherited environment. These controls reduce common blast radii but do not establish a complete hostile-code containment boundary. High-priority residual risks include writable bind-mounted source, Docker-daemon and runtime dependence, opt-in native execution, secret-bearing traceback locals, source disclosure through model prompts, untrusted model-generated replacement code, dependency-image drift, unbounded diagnostic output, and plaintext local history. The central security conclusion is architectural: human approval is necessary but insufficient. A credible repair system must combine visible intent, least privilege, data minimization, deterministic policy checks, isolated verification, provenance, and auditable state transitions. The paper concludes with a concrete security roadmap and testable acceptance criteria rather than claiming guarantees the prototype does not yet provide.

Keywords: AI-assisted debugging, threat modeling, sandboxing, prompt injection, secure code execution, human oversight, repair verification, data minimization

1. Scope and security posture

The assessed system is a single-user desktop prototype, not a multi-tenant execution service. Its normal workflow is: accept source, perform a language-specific check, execute it, capture output and selected Python exception state, send code and failure context to an optional model service, parse a structured diagnosis, preview a full-source replacement as a diff, require user approval, apply the proposed source in the editor, rerun it, and record the outcome.

The security claim is deliberately narrow. The repository implements defense-in-depth controls for local experimentation. It does not claim operating-system-grade isolation, protection from a compromised Docker daemon, safe execution of arbitrary hostile workloads, or confidentiality after a user explicitly chooses native execution. Security is therefore evaluated as risk reduction across boundaries, not as a binary “sandboxed / safe” label.

This model covers the Phase 6 Python implementation and its documented workflows. It excludes the security of the upstream model provider, container registry, compiler projects, operating system, and Docker implementation except where trust in those dependencies creates residual project risk. It also excludes physical compromise and malicious users who already possess unrestricted access to the same host account.

2. Method

The analysis uses an asset-and-boundary method with STRIDE-inspired threat categories: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. Because the product includes a generative model, the analysis also maps risks to prompt injection, sensitive-information disclosure, supply-chain exposure, misinformation, and excessive agency themes described by OWASP. NIST AI RMF and its Generative AI Profile provide the governance frame: risks should be mapped, measured, managed, and documented throughout the lifecycle. NIST SSDF informs the treatment of provenance, dependency integrity, testing, and vulnerability response.

Repository evidence was inspected at the implementation level. Security-relevant claims are tied to `core/sandbox.py`, `core/executor.py`, `core/ai_suggester.py`, `core/project.py`, `core/diagnostics.py`, `core/history.py`, `interface/gui.py`, CLI behavior, tests, workflow configuration, and README disclosures. When a control is absent, this paper labels it as a recommendation rather than quietly upgrading aspiration into fact.

Risk ratings combine likelihood and impact using ordinal values from 1 to 5. The product is the priority score: 1-4 low, 5-9 moderate, 10-16 high, and 17-25 critical. Ratings assume a developer workstation containing source code and ordinary user data, not production credentials or regulated records.

3. System and trust boundaries

3.1 Trust domains

Domain	Components	Primary trust assumption
Desktop host	GUI, CLI, Python process, filesystem, Docker client	The user account and application install are not already compromised.
Execution environment	Ephemeral container or explicit native subprocess	Container controls reduce reach; native execution receives host-user authority.
Model boundary	Prompt construction, API client, remote model, JSON response	Inputs and outputs are untrusted data; provider availability and policy are external dependencies.
Durable state	History JSON, saved source, saved regression tests	Local filesystem permissions and user decisions govern confidentiality and integrity.
Supply chain	Container images, Python packages, compilers, CI actions	Referenced artifacts may change or become compromised unless pinned and verified.

3.2 Data flow

- 1 Source enters through the editor, opened files, or CLI arguments.
- 2 The application copies source into an ephemeral execution workspace.
- 3 The selected backend executes the program and returns stdout, stderr, status, and optional Python trace locals.
- 4 For failed runs, source and diagnostic text may be inserted into a model prompt.
- 5 The model returns a JSON diagnosis containing explanation, confidence, full replacement source, and an optional regression test.
- 6 The application validates JSON shape and confidence range, renders a diff, and waits for approval.
- 7 Approval replaces editor content and triggers a new execution; rejection leaves source unchanged.
- 8 Runs and repair decisions are written to local history; users may separately save source or tests.

Every arrow crosses a security boundary. In particular, code is both data and executable content; diagnostic context may contain secrets; model output is both explanatory text and a candidate program; and a “verified” repair currently means only that the rerun returned success, not that the patch is secure or semantically correct.

4. Assets and security objectives

Asset	Confidentiality objective	Integrity objective	Availability objective
User source code	Do not expose beyond chosen execution and model boundaries.	Prevent silent or unauthorized modification.	Preserve recoverability and access.
Secrets in code, environment, files, and locals	Never inherit unnecessarily; redact before display, transmission, or storage.	Prevent adversarial substitution.	Avoid workflows that require secret disclosure.
Host filesystem and credentials	Keep untrusted programs away from host data and privileged services.	Prevent writes outside approved targets.	Prevent resource exhaustion or destructive modification.
Diagnosis and repair proposal	Protect proprietary context where applicable.	Bind proposal to the exact source and failure it analyzed.	Degrade safely when the model is unavailable.

Asset	Confidentiality objective	Integrity objective	Availability objective
Execution result and verification record	Avoid leaking sensitive output.	Make state, backend, code hash, and parent repair traceable.	Retain useful evidence without uncontrolled growth.
User intent and approval	Treat as a high-value authorization decision.	Ensure approval applies only to the visible, unchanged proposal.	Keep rejection and recovery reliable.
Build and runtime dependencies	Limit unnecessary disclosure.	Pin, verify, and update trusted artifacts.	Provide reproducible recovery from dependency failure.

The dominant security properties are least privilege, explicit authorization, provenance, isolation, data minimization, fail-closed behavior, and recoverability.

5. Adversaries and misuse cases

The primary adversary is hostile or compromised source code intentionally executed by the user. A second is a malicious repository that embeds instructions, oversized data, deceptive filenames, or secret-harvesting behavior. A third is an unreliable or adversarial model response that proposes subtly harmful code. A fourth is a compromised dependency, container image, model account, or local history file. Accidental misuse also matters: a user may enable native execution, approve a plausible diff too quickly, transmit confidential code, or mistake a zero exit status for correctness.

Representative goals include reading host files, stealing credentials, reaching the network, exhausting CPU/memory/processes/disk, modifying source, persisting across runs, manipulating model analysis through code comments or diagnostic text, causing unsafe repair approval, hiding changes in a large full-file replacement, poisoning history, and extracting sensitive locals into prompts or logs.

6. Implemented controls

6.1 Restricted Docker execution

The Docker backend is the default. The command disables container networking, caps memory at 256 MiB, limits CPU to 0.5, caps processes at 64, marks the container root filesystem read-only, drops all Linux capabilities, enables no-new-privileges, and supplies a bounded /tmp tmpfs with noexec and nosuid. Execution occurs in a fresh temporary directory that is removed after the run. The Docker client receives only a minimal PATH environment, which avoids blindly forwarding the application's API key and unrelated host variables.

These are substantive controls. Network denial blocks straightforward exfiltration from the container; resource controls reduce common denial-of-service paths; capability removal and privilege constraints narrow kernel-facing authority; and an ephemeral workspace reduces persistence.

The workspace is nevertheless bind-mounted into the container and is writable. The application copies only generated source and the trace harness into that temporary directory, so writes do not directly target the user's original project. However, a malicious program can tamper with files inside the run workspace, including trace artifacts, and can consume the allowed temporary storage. Container isolation also inherits the security of the host kernel, Docker daemon, image, and runtime configuration.

6.2 Fail-closed backend selection

If Docker is unavailable, the default path does not silently execute on the host. Environment-selected local execution requires an additional opt-in flag. This is an important boundary: convenience does not automatically erase isolation. The API also permits a caller to request backend="local" directly, so integrations must treat that argument as a privileged decision.

6.3 Human-governed repair

Model output is a proposal, not an automatic mutation. The diagnosis object separates summary, root cause, corrected source, explanation, regression test, confidence, and provider. The interface displays a diff and enables explicit approval or rejection. Only approval places replacement source into the editor and triggers a rerun. Rejections and verification outcomes are recorded.

This limits excessive agency, but approval is not a security proof. The model supplies complete replacement source, the UI does not yet enforce a patch-size threshold, and the rerun validates executable success rather than behavioral equivalence, security policy, or test-suite completion.

6.4 Structured model response and offline degradation

The model is instructed to return JSON and not claim verification. Required fields are checked and confidence is clamped to the interval [0,1]. Provider failures fall back to deterministic offline guidance rather than blocking core execution. These mechanisms improve robustness and reduce parser ambiguity. They do not establish that explanations are true, fixes are safe, or model-provided tests are trustworthy.

6.5 Bounded context collection

Project discovery accepts known source extensions, ignores common high-volume directories, stops at 40 files, and caps collected bytes at 250,000. Python local values captured on exceptions are represented defensively and truncated to 300 characters. These limits reduce cost and accidental bulk disclosure.

They are size controls, not sensitivity controls. Files may still contain credentials, proprietary algorithms, personal information, or malicious prompt instructions. Local-variable names are not screened, and exception context may reveal tokens, passwords, connection strings, or user data.

7. Threat register

ID	Threat	Existing control	L	I	Score	Priority
T1	Container escape or Docker/runtime compromise	Capability drop, no-new-privileges, read-only root, resource limits	2	5	10	High
T2	Native execution reads, writes, executes, or exfiltrates with user authority	Explicit opt-in and warning	4	5	20	Critical
T3	Secret disclosure through source, stderr, traceback locals, project context, or history	Minimal inherited environment; byte and representation bounds	4	5	20	Critical
T4	Prompt injection embedded in source/comments/output manipulates diagnosis	Human approval; output treated as proposal	4	4	16	High
T5	Harmful or deceptive model-generated replacement code	Diff preview, explicit approval, rerun	4	5	20	Critical
T6	"Verified" label overstates zero-exit-status evidence	Rerun and event logging	4	4	16	High
T7	Dependency or container-image compromise/drift	Named images and CI tests	3	5	15	High

ID	Threat	Existing control	L	I	Score	Priority
T8	Resource exhaustion through output, filesystem writes, compilation, or model usage	Timeout, memory/CPU/PID/tmpfs limits, token cap	3	4	12	High
T9	History tampering, disclosure, or unbounded retention	Local storage and event relationships	3	4	12	High
T10	Approval race or stale proposal applied to changed source	Pending original source retained for diff	3	5	15	High
T11	Malicious regression test saved or later executed as trusted code	Separate save action; not auto-executed	3	4	12	High
T12	Misleading path, symlink, or encoding behavior during project discovery and save	Resolved root; extension allowlist; user file dialogs	2	4	8	Moderate

7.1 Critical risk: native execution

Native execution is functionally equivalent to asking the current user account to run the submitted program. Timeouts limit duration, not authority. A program can read accessible files, use the network, spawn descendants, alter configuration, or persist before the timeout. The control is therefore informed consent, not containment. The GUI and CLI should present local execution as a separate hazardous mode, require per-session confirmation, show the exact interpreter/compiler command, and never enable it through an ambiguous inherited configuration.

7.2 Critical risk: diagnostic data disclosure

Source and error text are inserted directly into the model prompt. Traceback capture serializes local values. Project discovery can assemble many source files. None of these paths currently performs secret scanning, policy classification, user preview of outbound context, or field-level redaction. A private key, `.env` content copied into source, database URL, access token held in a local, or personal record in an exception message could cross the model boundary or persist in history.

The proper control is not merely a longer privacy notice. The application should minimize by default, display the exact outbound payload, redact high-risk patterns, permit file and frame exclusion, avoid collecting locals unless requested, and make remote diagnosis a distinct consent boundary.

7.3 Critical risk: unsafe repair acceptance

A well-formed JSON response may still contain a backdoor, data-loss behavior, weakened authentication, insecure dependency, hidden network request, or unrelated rewrite. Low temperature and confidence metadata do not transform probabilistic output into trusted code. The proposal should be bound cryptographically to the original code hash, error hash, model identifier, and creation time. Before approval, deterministic policy checks should flag new network calls, subprocess use, dynamic evaluation, secret access, permission changes, and unusually large diffs. After approval, verification should occur in a fresh restricted environment with declared tests.

8. Prompt-injection analysis

Direct prompt injection can appear inside source comments, string literals, exception messages, generated filenames, or project files. The model may be instructed to ignore debugging goals, leak prompt content, produce a broad rewrite, or insert malicious behavior. The current architecture reduces impact because the model cannot directly invoke tools and because mutation is approval-gated. This is a strong separation of recommendation from authority.

Residual risk remains social and semantic. A confident explanation can persuade the user; a large diff can hide behavior; and model-provided regression tests can be constructed to bless the proposed defect. Recommended mitigations are: delimit every untrusted field; label it as data; avoid mixing project content with system-level instructions; request minimal patches rather than complete files; reject responses outside a strict schema and size policy; compare security-sensitive APIs before and after; require independently derived tests; and render provenance beside every claim.

Prompt injection should be tested as an adversarial benchmark, not treated as a prompt-writing problem that can be solved once. The empirical protocol should include visible and encoded instructions in comments, exception text, dependency files, and multi-file context.

9. Verification and auditability gaps

The application records a repair as `verified_fix` when the replacement program returns success. This is useful operational evidence, but the term can be misunderstood. A successful exit does not prove the original bug is fixed, output is correct, tests pass, the patch preserves intent, or the patch is secure. Security language should distinguish four states:

- 1 **Executed:** the program completed under a named backend.
- 2 **Failure resolved:** the previously observed failure did not recur.
- 3 **Regression checked:** independent declared tests passed.
- 4 **Policy checked:** deterministic security rules found no prohibited change.

History entries should include source hashes, proposal hashes, backend and image digest, toolchain versions, command policy, limits, test hashes, result hashes, timestamps, and parent event. Append-only integrity protection would make unauthorized edits detectable. Sensitive content should be optional, redacted, encrypted at rest where appropriate, and governed by a retention policy.

10. Prioritized mitigation roadmap

P0 - before handling hostile or confidential projects

- Remove or redesign native execution for ordinary users. Require an unmistakable per-session hazard gate and never fall back to it.
- Add outbound-context preview and secret redaction for source, diagnostics, locals, and project files.
- Disable traceback-local capture by default; allow selective opt-in after showing the data boundary.
- Bind every proposal to hashes of the exact source and failure. Invalidate it whenever either changes.
- Replace full-file model output with bounded patch operations and reject oversized or path-expanding changes.
- Run approved repairs and regression checks in a fresh, non-networked environment rather than reusing mutable run state.

P1 - before broader beta distribution

- Pin container images by digest; generate a software bill of materials; scan images and Python dependencies; document update cadence.
- Add output, file-size, and disk quotas plus termination of descendant processes.
- Add deterministic rules for newly introduced network, process, dynamic-evaluation, credential, filesystem, and permission APIs.
- Encrypt or minimize history, add retention controls, and make deletion understandable and testable.
- Add a `SECURITY.md` with supported versions, disclosure channel, threat assumptions, and response expectations.

- Add security regression tests for backend selection, command construction, environment isolation, proposal invalidation, redaction, and history integrity.

P2 - research and hardening

- Evaluate rootless execution, user namespaces, seccomp/AppArmor profiles, gVisor, Firecracker, or a remote disposable worker depending on target deployment.
- Introduce signed provenance for builds and releases and verify downloaded artifacts.
- Measure approval quality under deceptive explanations, large diffs, prompt injection, and time pressure.
- Separate model-generated tests from independently generated or repository-owned tests.
- Establish incident logging that is useful without collecting raw secrets.

11. Security acceptance criteria

The next security milestone should be defined by tests rather than adjectives. A release may claim the hardened local profile only when: Docker absence fails closed; native execution cannot be enabled without a fresh visible decision; container network attempts fail; host secrets are absent from the container environment; CPU, memory, process, disk, output, and time limits are enforced; source changes invalidate pending approval; no proposal can alter a path outside the selected file set; outbound model context is previewable and redacted; images are digest-pinned; history retention is configurable; and verification records identify code, environment, tests, and policy results.

Adversarial tests should include fork bombs, output floods, disk floods, compiler bombs, symlink tricks, encoded prompt injection, malicious comments, fake trace markers, secret-bearing locals, dependency substitution, stale approvals, deceptive Unicode, enormous patches, tests that always pass, and repairs that suppress errors without correcting behavior.

12. Deployment profiles

Profile	Intended use	Required posture
Educational local	Small examples without secrets	Default Docker backend, no project-wide upload, clear limitations.
Developer workstation	Proprietary repositories	Redaction, context preview, pinned images, encrypted/minimized history, independent tests.
Hostile-code laboratory	Untrusted samples	Stronger isolation than ordinary Docker, disposable workers, no host mounts or credentials.
Multi-user service	Remote submissions	Out of current scope; requires tenancy isolation, authentication, quotas, monitoring, abuse response, and hardened orchestration.

The current implementation is closest to the educational-local profile. Marketing or documentation should not imply the other profiles until their acceptance criteria are met.

13. Limitations

This is a design-level and repository-level threat model, not a penetration test, formal verification, dependency audit, or Docker escape assessment. Risk scores are prioritization aids and will change with deployment context. The review did not assume undisclosed infrastructure or controls. Model-provider retention, jurisdiction, training usage, and contractual guarantees must be evaluated against the actual account and service configuration before confidential use.

The model should be updated whenever execution architecture, context collection, provider integration, persistence, plugin/tool authority, distribution, or deployment topology changes. A threat model that never changes is just a fossil with headings.

14. Conclusion

AI Debugger Pro already contains the most important architectural instinct for AI-assisted repair: the model proposes, the human decides, and execution supplies evidence. The Docker defaults, fail-closed behavior, minimal inherited environment, structured output, and explicit approval gate are meaningful controls. They reduce risk without pretending that probabilistic diagnosis is authority.

The remaining work is equally clear. Native execution must be treated as hazardous; outbound context must be minimized and previewed; repair proposals must be bound to immutable inputs; verification must become layered; dependencies must be pinned; and durable records must protect both privacy and integrity. With those changes, the project can evolve from a careful prototype into a defensible research platform for human-governed repair. The right security story is not “the box is unbreakable.” It is “every boundary is visible, every authority is narrow, every change is attributable, and every claim is testable.”

Appendix A. Control-to-code evidence

Control or exposure	Repository evidence
Docker is the default backend	core/sandbox.py::run_sandboxed
Network denial, resource limits, read-only root, capability drop	core/sandbox.py::_docker_command
Minimal environment passed to Docker client	core/sandbox.py::run_sandboxed
Explicit local-execution gate	core/sandbox.py::run_sandboxed; tests/test_sandbox.py
Trace-local truncation	core/sandbox.py::_trace_harness
Source and error sent in prompt	core/ai_suggester.py::ai_diagnose
Structured response parsing	core/ai_suggester.py::_parse_diagnosis; core/diagnostics.py
Proposal preview and approval	interface/gui.py::_receive_diagnosis; apply_and_verify
Bounded project discovery	core/project.py::load_project
Durable local execution history	core/history.py::ExecutionHistory

Appendix B. References

- 1 National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- 2 National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- 3 National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1*. NIST SP 800-218, 2022. <https://doi.org/10.6028/NIST.SP.800-218>
- 4 OWASP Foundation. *Top 10 for Large Language Model Applications, 2025*. <https://genai.owasp.org/llm-top-10/>
- 5 MITRE. *Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)*. <https://atlas.mitre.org/>
- 6 Docker, Inc. *Docker Engine security documentation*. <https://docs.docker.com/engine/security/>
- 7 T. R. Bentley. *AI Debugger Pro: Current Architecture and Human-Governed Repair Protocol*. September 2026.
- 8 T. R. Bentley. *Empirical Evaluation Protocol for Human-Governed AI-Assisted Repair*. September 2026.